

BEGINNER GUIDE · PYTHON · AGENT STRUCTURED OUTPUT

에이전트를 위한 Python 타입과 Pydantic

Enum, Literal, TypedDict, Annotated, ConfigDict와 복잡한 Structured Output 설계를
한 흐름으로 이해하는 입문서

TYPE	VALIDATE	SCHEMA	AGENT
------	----------	--------	-------

프로젝트 예제: baseline/PoC/2_embedding의 기업 관계 추출 구조
작성 기준: Python 3.10+ · Pydantic v2 · 2026-07-18

먼저 큰 그림부터

에이전트의 출력 구조를 설계할 때 가장 먼저 구분해야 하는 것은 타입을 설명하는 도구와 실행 중 값을 검사하는 도구입니다. 이 둘을 섞어 이해하면 코드가 그럴듯해 보여도 잘못된 데이터가 그대로 통과할 수 있습니다.

도구	핵심 역할	실행 중 검증	에이전트에서 주 용도
typing	값의 예상 타입을 표현	기본적으로 하지 않음	IDE, 타입 검사기, 스키마 재료
Enum / Literal	허용 가능한 선택지를 제한	단독 사용 시 보장하지 않음	관계명, 상태, 분류값
TypedDict	dict의 키와 값 모양을 설명	하지 않음	내부 상태, 함수 간 데이터 계약
Annotated	타입에 추가 메타데이터 부착	사용하는 라이브러리에 따라 다름	Field, 제약, discriminator
Pydantic	입력을 파싱하고 검증하며 직렬화	함	LLM 출력, API, 설정, 도구 입력

한 문장 요약

typing은 설계도이고, Pydantic은 입구의 검사대입니다. 에이전트에서는 둘을 함께 사용해 LLM 응답을 읽을 수 있고 검증 가능한 객체로 바꿉니다.

에이전트 데이터가 지나가는 경로

1. 텍스트 입력	2. LLM 추출	3. 스키마 검증	4. 의미 검증	5. 저장 / 도구 실행
-----------	-----------	-----------	----------	---------------

Pydantic은 3번을 담당합니다. 4번의 온톨로지 규칙이나 사실성까지 자동으로 보장하지는 않으므로 별도의 검증 로직이 필요합니다.

02

Python 타입 표기의 기본

타입 힌트는 사람이 코드를 읽기 쉽게 만들고 IDE와 정적 타입 검사기가 오류를 찾도록 돕습니다. 그러나 Python 런타임은 일반적인 타입 힌트를 자동으로 강제하지 않습니다.

```
def normalize_company(name: str, aliases: list[str]) -> str:
    return name.strip()

# Python
normalize_company(123, " ")
```

리스트, 딕셔너리, 선택적 값

```
company_names: list[str] = ["", "SK "]
scores: dict[str, float] = {"": 0.92}
company_id: int | str = "005930"
note: str | None = None
```

`|` 기호는 무엇인가?

Python 3.10부터 `A | B`는 `A` 또는 `B`이라는 뜻입니다. 이것은 값의 후보를 나열하는 문법이 아니라 타입의 합집합입니다.

표현	뜻	주의점
<code>int str</code>	정수 또는 문자열	두 타입 모두 허용
<code>str None</code>	문자열 또는 None	<code>Optional(str)</code> 와 같은 뜻
<code>Literal('buy', 'sell')</code>	정확히 두 문자열 값 중 하나	값 선택지는 <code>Literal</code> 사용
<code>'buy' 'sell'</code>	잘못된 표현	문자열 값끼리는 <code> </code> 를 사용할 수 없음

Optional의 흔한 함정

`value: float | None`은 None을 허용하지만 필드 자체는 여전히 필수일 수 있습니다. 기본값까지 선택적으로 만들려면 `value: float | None = None`으로 씁니다.

03

Enum과 Literal: 선택지를 닫는 법

에이전트가 자유 문자열을 반환하게 두면 같은 뜻이 여러 철자로 나타날 수 있습니다. 예: partner, PARTNERED_WITH, partnership. 선택지를 닫으면 후속 코드가 단순해집니다.

Enum

```
from enum import Enum

class RelationType(str, Enum):
    MENTIONS = "MENTIONS"
    ANNOUNCED = "ANNOUNCED"
    PARTNERED_WITH = "PARTNERED_WITH"
    LOCATED_IN = "LOCATED_IN"
    ANALYZED_BY = "ANALYZED_BY"
```

- 이름을 붙여 여러 모델과 함수에서 재사용하기 좋습니다.
- 멤버를 순회하거나 문서화하기 쉬워 도메인 용어집에 적합합니다.
- Pydantic 모델 안에서 사용하면 입력값 검증과 JSON Schema 생성에 참여합니다.

Literal

```
from typing import Literal

Decision = Literal["buy", "hold", "sell"]

class MentionEdge(BaseModel):
    kind: Literal["mention"]
    evidence: str
```

- 선택지가 짧고 한두 곳에서만 사용될 때 간결합니다.
- 서로 다른 모델을 구분하는 discriminator 필드에 특히 잘 맞습니다.

상황	추천
관계 타입처럼 프로젝트 전반에서 공유되는 용어	Enum
상태값이 2~3개이며 한 모델에서만 사용	Literal
서로 다른 모델을 kind 필드로 구분	Literal + discriminated union
int 또는 str처럼 타입 자체가 여러 개	A B

TypedDict: dict의 모양을 설명

TypedDict는 일반 dict가 어떤 키를 가져야 하고 각 값이 무슨 타입이어야 하는지 타입 검사기에게 알려줍니다. 실행 중 결과는 여전히 평범한 dict입니다.

```
from typing import NotRequired, TypedDict

class AgentState(TypedDict):
    company: str
    article_ids: list[str]
    extracted_count: int
    warning: NotRequired[str]

state: AgentState = {
    "company": " ",
    "article_ids": ["news-001", "news-002"],
    "extracted_count": 4,
}
```

좋은 사용처

- 이미 dict를 중심으로 동작하는 에이전트 상태
- 함수 사이에서 전달되는 가벼운 내부 데이터
- 검증이 끝난 Pydantic 모델을 `model_dump()`로 바꾼 뒤의 타입 표현

적합하지 않은 사용처

- 외부 API 입력, 파일, LLM 응답처럼 신뢰할 수 없는 데이터
- 값 변환, 상세 오류 메시지, JSON Schema가 필요한 경계

질문	TypedDict	Pydantic BaseModel
실제 객체	dict	모델 인스턴스
런타임 검증	없음	있음
타입 검사기 지원	좋음	좋음
JSON Schema 생성	직접 제공하지 않음	<code>model_json_schema()</code>
직렬화	이미 dict	<code>model_dump()</code> , <code>model_dump_json()</code>

실전 기준

경계에서는 Pydantic으로 검사하고, 내부에서 dict가 편하면 TypedDict로 표현하세요. 둘 중 하나만 골라야 하는 경쟁 관계가 아닙니다.

05

Annotated: 타입에 의미를 덧붙이기

`Annotated[T, metadata]`는 기본 타입 `T`에 라이브러리가 읽을 추가 정보를 붙이는 표준 문법입니다. Python 자체는 대체로 `T`처럼 다루지만, Pydantic은 `Field` 제약과 `discriminator` 같은 메타데이터를 읽습니다.

```
from typing import Annotated
from pydantic import BaseModel, Field

Confidence = Annotated[
    float,
    Field(ge=0.0, le=1.0, description="0 1 ")
]

class RelationClaim(BaseModel):
    confidence: Confidence
```

같은 뜻을 표현하는 두 방식

```
class A(BaseModel):
    confidence: float = Field(ge=0.0, le=1.0)

class B(BaseModel):
    confidence: Annotated[float, Field(ge=0.0, le=1.0)]
```

간단한 필드는 첫 번째가 읽기 쉽습니다. 여러 모델에서 반복되는 제약을 재사용하거나 union 전체에 `discriminator`를 붙일 때는 `Annotated`가 더 자연스럽습니다.

복잡한 구조에서 가장 중요한 패턴

```
class MentionEdge(BaseModel):
    kind: Literal["mention"]
    target: str

class PartnershipEdge(BaseModel):
    kind: Literal["partnership"]
    partner: str

Edge = Annotated[
    MentionEdge | PartnershipEdge,
    Field(discriminator="kind"),
]
```

왜 `discriminator`가 필요한가?

Pydantic이 union의 모든 모델을 추측하며 검사하지 않고, `kind` 값을 보고 정확한 모델을 바로 선택하게 합니다. 오류 메시지와 JSON Schema도 더 명확해집니다.

06

Pydantic BaseModel: 검증 가능한 데이터 계약

Pydantic 모델은 Python 타입 힌트를 실제 검증 규칙으로 사용합니다. 또한 중첩 모델, 값 변환, 오류 위치, 직렬화, JSON Schema 생성을 한곳에서 제공합니다.

```
from pydantic import BaseModel, Field

class Company(BaseModel):
    id: int | str = Field(description=" ")
    name: str = Field(min_length=1, description=" ")
    aliases: list[str] = Field(default_factory=list)

company = Company.model_validate({
    "id": "005930",
    "name": " ",
})
```

자주 쓰는 함수

함수	사용 목적
Model.model_validate(data)	dict나 객체를 검증해 모델 인스턴스로 변환
Model.model_validate_json(text)	JSON 문자열을 바로 검증
model.model_dump()	모델을 Python dict로 변환
model.model_dump(mode='json')	JSON 호환 값으로 변환
model.model_dump_json()	JSON 문자열로 직렬화
Model.model_json_schema()	LLM 또는 API에 전달할 JSON Schema 생성

Field가 하는 일

```
class Entity(BaseModel):
    entity_id: str = Field(
        min_length=1,
        description=" ID",
    )
    name: str = Field(description=" ")
    confidence: float = Field(ge=0.0, le=1.0)
```

- description은 LLM이 필드 의미를 이해하는 데 도움을 줍니다.
- min_length, ge, le 같은 제약은 응답을 받은 뒤 실제로 검사됩니다.
- default_factory=list는 모든 인스턴스가 안전하게 독립된 빈 리스트를 갖게 합니다.

ConfigDict와 extra='forbid'

ConfigDict는 Pydantic v2 모델의 동작 방식을 정합니다. Structured Output에서는 모델이 스키마에 없는 필드를 만들어도 조용히 사라지지 않도록 extra='forbid'를 권장합니다.

```
from pydantic import BaseModel, ConfigDict

class StrictModel(BaseModel):
    model_config = ConfigDict(
        extra="forbid",
        str_strip_whitespace=True,
    )

class Entity(StrictModel):
    name: str
```

extra의 세 가지 모드

설정	알 수 없는 필드가 들어오면	에이전트 출력에서의 의미
extra='ignore'	버림	오류를 숨길 수 있음
extra='allow'	보존	출력 모양이 예측하기 어려워짐
extra='forbid'	ValidationError	스키마 위반을 즉시 발견

```
Entity.model_validate({
    "name": " ",
    "ticker": "005930", #
})
# ValidationError: Extra inputs are not permitted
```

strict=True와는 다른 개념

extra='forbid'는 필드 이름을 엄격하게 검사합니다. ConfigDict(strict=True)는 문자열 '1'을 숫자 1로 바꾸는 식의 타입 강제 변환을 줄입니다. 둘은 목적이 다르며 함께 사용할 수도 있습니다.

권장 기본 클래스

여러 Structured Output 모델이 같은 정책을 써야 한다면 StrictModel 하나를 만들고 모든 모델이 상속하게 하세요. 설정 누락과 중복을 줄일 수 있습니다.

복잡한 Structured Output 설계 원칙

복잡한 출력은 큰 dict 하나로 만들기보다 의미 단위로 작은 모델을 조합해야 합니다. 다음 원칙은 스키마 오류뿐 아니라 후속 처리의 모호함도 줄여 줍니다.

원칙	구조	이유
반복되는 레코드	<code>list(ChildModel)</code>	각 항목의 필드 대응 관계가 유지됨
달린 선택지	Enum 또는 Literal	오탈자와 임의 표현을 차단
형태가 여러 종류	discriminated union	종류별 필드를 안전하게 분리
미정 값	<code>T None = None</code>	없음과 빈 문자열을 구분
중첩 객체	별도 BaseModel	재사용, 검증, 오류 위치가 명확
추가 필드	<code>extra='forbid'</code>	모델이 만든 예상 밖 필드를 발견

피해야 할 병렬 리스트

```
class CompanyRelation(BaseModel):
    relations: list[str]
    evidence: list[str]
```

이 구조에서는 `relations[0]`이 어떤 `evidence`와 연결되는지 오직 인덱스에 의존합니다. 두 리스트의 길이가 다르거나 순서가 어긋나면 관계와 근거가 잘못 결합됩니다.

대신 하나의 레코드로 묶기

```
class RelationClaim(BaseModel):
    relation: RelationType
    evidence: list[str]
    confidence: float = Field(ge=0.0, le=1.0)

class CompanyRelation(BaseModel):
    object_name: str
    claims: list[RelationClaim]
```

구조 설계 질문

두 값이 항상 함께 움직여야 한다면 같은 모델에 넣으세요. 리스트의 같은 인덱스에 우연히 놓여 있어야만 연결되는 구조는 장기적으로 깨지기 쉽습니다.

PoC/2_embedding에 적용한 개선 예제

현재 PoC는 기업별 관계와 근거를 추출합니다. 아래 구조는 기존 목적을 유지하면서 온톨로지 선택지를 닫고, 관계와 근거를 한 레코드로 묶고, 예상 밖 필드를 금지합니다.

1단계: 공통 정책과 닫힌 용어집

```
from enum import Enum
from typing import Annotated, Literal
from pydantic import BaseModel, ConfigDict, Field

class StrictModel(BaseModel):
    model_config = ConfigDict(
        extra="forbid",
        str_strip_whitespace=True,
    )

class EntityType(str, Enum):
    NEWS_ARTICLE = "NewsArticle"
    ORGANIZATION = "Organization"
    PERSON = "Person"
    PLACE = "Place"
    PRODUCT = "Product"

class RelationType(str, Enum):
    MENTIONS = "MENTIONS"
    ANNOUNCED = "ANNOUNCED"
    PARTNERED_WITH = "PARTNERED_WITH"
    LOCATED_IN = "LOCATED_IN"
    ANALYZED_BY = "ANALYZED_BY"
```

2단계: 엔티티와 근거

```
class Evidence(StrictModel):
    sentence: str = Field(min_length=1)
    source_id: str | None = None

class Entity(StrictModel):
    entity_id: str = Field(min_length=1)
    name: str = Field(min_length=1)
    entity_type: EntityType
    aliases: list[str] = Field(default_factory=list)
```

엔티티를 별도 모델로 두면 이후 그래프 저장, 중복 제거, 별칭 통합 단계에서도 같은 데이터 계약을 재사용할 수 있습니다.

09B

PoC 개선 예제 - 관계 모델

관계 종류마다 필요한 필드가 달라질 가능성이 있다면 discriminated union을 사용합니다. 모든 관계가 완전히 같은 모양이라면 단일 RelationClaim 모델만으로도 충분합니다.

```
class MentionClaim(StrictModel):
    kind: Literal["mention"]
    relation: Literal[RelationType.MENTIONS]
    target_id: str
    evidence: list[Evidence] = Field(min_length=1)

class GraphRelationClaim(StrictModel):
    kind: Literal["graph_relation"]
    relation: RelationType
    source_id: str
    target_id: str
    evidence: list[Evidence] = Field(min_length=1)
    confidence: float = Field(ge=0.0, le=1.0)

Claim = Annotated[
    MentionClaim | GraphRelationClaim,
    Field(discriminator="kind"),
]

class ExtractionResult(StrictModel):
    focus_company: str
    entities: list[Entity]
    claims: list[Claim]
```

LangChain에서 사용

```
structured_llm = llm.with_structured_output(
    ExtractionResult,
    method="json_schema",
    strict=True,
    include_raw=True,
)

response = (prompt | structured_llm).invoke(inputs)

if response["parsing_error"] is not None:
    raise response["parsing_error"]

result: ExtractionResult = response["parsed"]
```

include_raw=True의 장점

파싱에 실패했을 때 원래 모델 응답과 parsing_error를 함께 관찰할 수 있어 프롬프트와 스키마를 디버깅하기 쉽습니다. 사용하는 LangChain 및 모델 조합에서 지원 여부는 확인하세요.

스키마 검증과 의미 검증을 분리

Structured Output과 Pydantic이 성공했다고 해서 추출 내용이 사실이거나 온톨로지 규칙에 맞는 것은 아닙니다. 검증을 두 층으로 나누면 책임과 오류 원인이 분명해집니다.

검증 층	확인하는 것	예시
구조 검증	필드 존재, 타입, 범위, 추가 필드	confidence가 0~1인지
의미 검증	도메인 규칙, 참조 무결성, 사실 근거	LOCATED_IN의 target이 Place인지

온톨로지 규칙 검증 예시

```
ALLOWED_ENDPOINTS = {
    RelationType.ANNOUNCED:
        (EntityType.ORGANIZATION, EntityType.PRODUCT),
    RelationType.PARTNERED_WITH:
        (EntityType.ORGANIZATION, EntityType.ORGANIZATION),
    RelationType.LOCATED_IN:
        (EntityType.ORGANIZATION, EntityType.PLACE),
}

def validate_relation(
    claim: GraphRelationClaim,
    entities: dict[str, Entity],
) -> None:
    source = entities[claim.source_id]
    target = entities[claim.target_id]
    expected = ALLOWED_ENDPOINTS.get(claim.relation)

    if expected and (
        source.entity_type,
        target.entity_type,
    ) != expected:
        raise ValueError("    domain/range    ")
```

Validator를 어디에 둘까?

- 필드 하나의 범위나 형식은 Field 또는 field_validator에 둡니다.
- 한 모델 안의 여러 필드 관계는 model_validator가 적합합니다.
- 외부 온톨로지, 데이터베이스, 다른 객체가 필요한 검증은 별도 도메인 함수에 둡니다.

경계의 원칙

Pydantic 모델은 가능한 한 자체 데이터만 검증하게 하고, 외부 상태가 필요한 규칙은 서비스 계층으로 분리하세요. 테스트와 재사용이 쉬워집니다.

자주 발생하는 실수

실수	문제	수정 방향
typing만 쓰고 검증했다고 생각	잘못된 런타임 값이 통과	외부 경계에서 model_validate
모든 선택지를 str로 둬	오탈자와 표현 흔들림	Enum 또는 Literal
모든 필드를 Optional로 둬	필수 데이터 누락을 발견 못함	정말 없어도 되는 값만 None
list(str) 여러 개로 대응 관계 표현	길이와 순서 불일치	ChildModel 리스트로 묶기
extra 기본 동작에 의존	예상 밖 필드가 조용히 버려질 수 있음	extra='forbid'
Any를 넓게 사용	스키마와 타입 검사 효과 감소	중첩 모델 또는 제한된 union
거대한 단일 모델	변경과 테스트가 어려움	의미 단위로 작은 모델 조합
스키마 성공을 사실성으로 착각	근거 없는 추출을 신뢰	의미 검증과 평가 추가

설계 전 체크리스트

- 이 필드는 필수인가, 아니면 None이 허용되는가?
- 선택지가 닫혀 있다면 Enum 또는 Literal로 표현했는가?
- 항상 함께 움직이는 값들이 같은 하위 모델에 들어 있는가?
- 종류별 구조가 다르다면 discriminator 필드가 있는가?
- 추가 필드를 금지했는가?
- Field description이 LLM에게 충분히 구체적인가?
- 구조 검증 뒤에 별도의 의미 검증이 있는가?
- 실패 시 raw 응답과 오류를 기록해 원인을 볼 수 있는가?

가장 작은 안전한 구조부터

처음부터 모든 가능성을 담은 거대 스키마를 만들지 마세요. 현재 후속 단계가 실제로 필요로 하는 필드만 엄격하게 정의하고, 새로운 요구가 생길 때 확장하는 편이 좋습니다.

추천 학습 순서

한꺼번에 모든 typing 문법을 외우기보다, 현재 PoC를 조금씩 강화하는 순서로 배우면 각 개념의 필요성이 자연스럽게 보입니다.

단계	배울 내용	PoC에 적용할 작은 과제
1	기본 타입, list, dict, T None	함수 입출력에 타입 힌트 추가
2	Literal, Enum	relation과 entity_type 선택지 달기
3	BaseModel, Field	CompanyRelation을 검증 가능한 모델로 만들기
4	ConfigDict, extra='forbid'	예상 밖 필드에 실패하는 테스트 작성
5	TypedDict	검증 후 에이전트 내부 상태 타입 정의
6	Annotated, discriminated union	서로 다른 claim 종류 분리
7	validator와 도메인 함수	온톨로지 domain/range 검사
8	JSON Schema와 Structured Output	스키마와 raw 응답을 함께 관찰

권장 연습

- 정상 입력 1개, 필수 필드 누락 1개, extra 필드 1개를 model_validate로 시험합니다.
- Literal에 없는 관계명을 넣어 ValidationError의 위치를 읽어 봅니다.
- relations/evidence 병렬 리스트를 claims: list[RelationClaim]으로 바꿔 결과를 비교합니다.
- model_json_schema() 결과에서 required, enum, additionalProperties를 찾아봅니다.

빠른 복습 공식

TYPE → SHAPE → VALIDATE → MEANING

typing으로 타입을 표현하고, 작은 모델로 모양을 만들고, Pydantic으로 구조를 검증한 뒤, 온톨로지와 근거 규칙으로 의미를 검증합니다.

공식 문서와 추천 영상

공식 문서는 필요한 개념을 확인할 때 가장 정확한 기준입니다.

Python typing

Python 한국어 typing 문서

타입 힌트가 런타임에서 자동 강제되지 않는다는 점부터 TypedDict, Annotated까지 확인합니다.

mypy Python 3 cheatsheet

자주 쓰는 타입 표기를 빠르게 훑고 예제를 따라 하기 좋습니다.

Pydantic v2

Pydantic 공식 문서 시작점

Models · Fields · Configuration · Unions · Validators

에이전트 Structured Output

LangChain Structured Output

OpenAI Structured Outputs 가이드

영상과 강의

Python Types for Fun and Profit - PyCon US 2022

타입 힌트를 기초부터 실습 중심으로 익힐 수 있는 긴 튜토리얼입니다.

Type safe data validation using Pydantic v2 - PyBay 2023

Pydantic v2, Annotated, 정적 타입과 런타임 검증의 관계를 다룹니다.

Pydantic for LLM Workflows - DeepLearning.AI

LLM 응답과 도구 입력을 Pydantic으로 검증하는 에이전트 응용 중심 과정입니다.

문서를 읽는 순서

Python typing 문서에서 기본 타입과 union을 익힌 뒤, Pydantic Models와 Fields를 실습하세요. 그 다음 Configuration과 Unions를 보고 Structured Output 문서로 넘어가면 좋습니다.

한 페이지 치트시트

필요	표현
문자열 리스트	<code>list(str)</code>
키가 문자열이고 값이 실수인 dict	<code>dict(str, float)</code>
문자열 또는 None	<code>str None</code>
선택적 필드와 기본값	<code>str None = None</code>
짧은 고정 선택지	<code>Literal['a', 'b']</code>
재사용할 고정 선택지	<code>class Kind(str, Enum): ...</code>
dict 내부 구조 설명	<code>class State(TypedDict): ...</code>
런타임 검증 모델	<code>class Result(BaseModel): ...</code>
필드 설명과 범위	<code>Field(description='...', ge=0, le=1)</code>
타입에 제약 메타데이터	<code>Annotated(float, Field(ge=0, le=1))</code>
여러 모델 중 하나	<code>A B</code>
구분 가능한 여러 모델	<code>Annotated(A B, Field(discriminator='kind'))</code>
추가 필드 금지	<code>ConfigDict(extra='forbid')</code>
dict 검증	<code>Model.model_validate(data)</code>
JSON 문자열 검증	<code>Model.model_validate_json(text)</code>
모델을 dict로	<code>model.model_dump()</code>
JSON Schema 생성	<code>Model.model_json_schema()</code>

최종 판단표

질문	예	아니오
외부에서 들어오는 데이터인가?	Pydantic으로 검증	typing만으로도 충분할 수 있음
선택지가 고정되어 있는가?	Enum / Literal	일반 타입
종류에 따라 필드가 달라지는가?	discriminated union	단일 모델
항상 함께 움직이는 값인가?	하위 모델 하나로 묶기	독립 필드 가능
외부 규칙을 확인해야 하는가?	별도 의미 검증	Pydantic 제약으로 충분

핵심

좋은 Structured Output은 에이전트와 후속 코드가 같은 의미를 공유하도록 검증 가능한 데이터 계약을 설계하는 일입니다.